Oracle Cloud Infrastructure

Behavioral Interview Guide: STAR Answers for Every OCI Core Value

Dev Kumar | Principal Software Engineer, Core Infrastructure (Job ID 345784, Nashville, TN)

How to use this guide

This guide covers all ten OCI core values from the interview prep pack, with three behavioral questions per value (30 in total), including the four sample questions OCI lists in the pack. Every answer follows the STAR format OCI asks for and is built from your resume: Sprouts AI, Resilience Inc, Tata Steel, and your Keystone project.

- Aim for 2 to 3 minutes per answer. Spend about 20% on Situation and Task, 60% on Action, and 20% on Result.
- Use "I", not "we", for the Action. OCI's pack says interviewers want to know what *you* did and why.
- End with a number and a lesson. The resume metrics are your strongest evidence; use them.
- Expect deep technical follow-ups. Each answer lists likely probes. At Principal level, interviewers will drill into the distributed-systems details (quorum, failover, retries, idempotency).

> **Before the interview:** The headline metrics come straight from your resume. Supporting details such as timelines, team sizes, page counts and incident durations are realistic examples consistent with that resume. Replace any that differ from what actually happened with your real figures, so every detail holds up when an interviewer probes it.

Your story bank at a glance

Prepare these stories well; each one can answer several values if you change the emphasis.

Story	Company	Values it can answer
Hot-shard latency fix for enterprise go-live	Sprouts AI	Customers First, Act Now, Take Pride
Zero-downtime online schema migration	Sprouts AI	Customers First, Nail the Basics
Active-passive vs active-active recommendation	Resilience Inc	Customers First, Challenge Ideas
Alert-noise cleanup and SLO burn-rate alerts (MTTR -45%)	Sprouts AI	Act Now, Nail the Basics

Story	Company	Values it can answer
Fault-injection and brownout testing program	Sprouts AI	Take Risks, Take Pride
Network partition incident (incident commander)	Sprouts AI	Remain Calm, Earn Trust
Aggressive failover timeout that caused flapping	Sprouts AI	Take Risks (failure), Own Without Ego
Retry-amplification challenge and retry budgets	Sprouts AI	Challenge Ideas, Nail the Basics
3x throughput, p99 under 40 ms, cost flat	Sprouts AI	Challenge Ideas, Take Pride
New OCI region in five weeks via Terraform modules	Sprouts AI	Embrace Change, Champion Execution
JVM / GC tuning instead of rewrite (p99 -55%)	Resilience Inc	Nail the Basics, Challenge Ideas
Idempotent Kafka pipeline (2B+ events/day)	Resilience Inc	Nail the Basics, Take Pride
Critical CVE and audit re-prioritization	Resilience Inc	Embrace Change, Customers First
Redis cache invalidation bug	Tata Steel	Own Without Ego, Take Risks
Patching automation (ops tickets -30%)	Tata Steel	Act Now, Own Without Ego
Leading the CI/CD transition	Tata Steel	Embrace Change, Innovate Together
Keystone Raft store correctness under 41 faults	Personal project	Take Pride, Nail the Basics

Contents

OCI core value	Questions
1. Put Customers First	Q1 – Q3
2. Act Now, Iterate	Q4 – Q6
3. Nail the Basics	Q7 – Q9
4. Expect and Embrace Change	Q10 – Q12
5. Innovate Together	Q13 – Q15
6. Take Risks, Remain Calm	Q16 – Q18
7. Own Without Ego	Q19 – Q21
8. Earn Trust, Give Trust	Q22 – Q24
9. Take Pride in Your Work	Q25 – Q27
10. Challenge Ideas, Champion Execution	Q28 – Q30
Prep-pack question map, questions to ask, final reminders	Last pages

1. Put Customers First

What it means: OCI wants engineers who listen carefully, respond fast, advise honestly, and do what is right for the customer even when it differs from what they literally asked for, or from what is easiest for the team.

Interviewers look for: direct customer contact, trade-offs where the customer won over internal convenience, and honest pushback that served the customer better.

Q1. Tell me about a time you had to prioritize your customers' needs. What did you do, how did it go, and what would you do differently knowing what you know now?

From the OCI prep pack

Situation: At Sprouts AI, a large enterprise tenant was two weeks from go-live on our multi-tenant retrieval platform on OCI. During their pre-production load tests, their p99 retrieval latency jumped from about 35 ms to over 300 ms every morning peak. Their launch date was tied to a public announcement, so a slip would have cost them credibility.

Task: I owned the data-plane retrieval path, so my goal was to find the root cause and get them back under our 40 ms p99 SLO before go-live, without hurting the other tenants on the same fleet.

Action: I paused my roadmap work and joined daily calls with their engineering team so I could hear exactly how they were driving traffic. Using our storage-node metrics I found their key design was concentrating writes and reads on a few partitions, creating hot shards. As a short-term fix I shipped hot-key detection with an adaptive cache and split the hottest partitions within four days, behind a feature flag and a canary on one cell first. In parallel I worked with their team on a better key-salting scheme, and I wrote the design for load-aware partition rebalancing as the long-term fix.

Result: Their p99 returned to under 40 ms, they went live on schedule, and within the next quarter they roughly doubled their usage. The rebalancer we built afterwards removed this whole class of issue for every tenant. What I would do differently: we only found it because the customer complained. I would have per-tenant latency dashboards and alerts in place from day one, which is exactly what I built right after this, so we now see a tenant's degradation before they do.

Likely follow-ups:

- Why not just add more nodes for that tenant?
- How did you make sure the fix did not affect other tenants?
- How did you communicate progress to the customer?

Q2. Tell me about a time you did what was right for the customer instead of exactly what they asked for.

Situation: At Resilience Inc, an enterprise customer of our multi-region event platform on Azure and Google Cloud asked us to make their deployment active-active with writes accepted in both regions. Their goal was zero downtime during a regional outage.

Task: As the engineer leading availability design, I had to decide whether to build what they asked for or recommend something different that actually met their underlying goal.

Action: I first dug into their workload and found that their events had strict per-account ordering requirements. Active-active writes would have required cross-region conflict resolution, which introduces subtle ordering bugs and data inconsistencies, the opposite of what they wanted. I wrote a short proposal that compared both options against their real RTO and RPO targets, and recommended active-passive with replicated state, health-checked automatic failover, and a recovery-oriented runbook. I walked their architects through the trade-offs honestly, including what we would give up, and committed to quarterly failover drills they could observe.

Result: They agreed. We hit 99.95% availability, and our failover drills consistently completed in about 90 seconds with no data loss. A few months later a real regional incident hit and their traffic failed over in under two minutes with no ordering issues. Their architect told us that the honest pushback was the reason they trusted us with a second workload.

Likely follow-ups:

- What if the customer had insisted on active-active?
- How did you quantify RTO and RPO?

Q3. Describe a situation where the easy path for your team conflicted with what was best for customers.

Situation: At Sprouts AI we needed to migrate the storage metadata schema used by our data-plane services. The quickest option was a hard cutover during a four-hour maintenance window, which several engineers preferred because it was simpler to build and test.

Task: I was leading the design, and I had to decide whether the engineering convenience justified asking every tenant to accept downtime on a platform with a 99.99% availability target.

Action: I argued that a four-hour window alone would consume more than our entire annual 99.99% error budget, and that customers would have to coordinate their own outages around ours. I proposed an online migration: dual-write to both schemas, backfill in throttled batches, verify with checksums and shadow reads, then flip reads per cell with an instant rollback flag. It added about three weeks of work, so I split it into milestones and took the riskiest piece, the shadow-read verifier, myself.

Result: We migrated the full fleet with zero customer-facing downtime and no customer action required. The pattern became our standard playbook and is a big part of how we have since shipped more than 300 in-service updates without downtime.

Likely follow-ups:

- How did you convince the engineers who wanted the cutover?
- What did the verification find?

2. Act Now, Iterate

What it means: Bias for action: fix what is broken, fill gaps, improve bad processes, and ship a simple working solution now rather than debating a perfect one. Offer solutions, not problem statements.

Interviewers look for: self-started fixes, a scrappy first version followed by deliberate iteration, and data used to decide the next step.

Q4. Tell me about a time you noticed something that needed fixing and fixed it without being asked.

Situation: At Tata Steel, OS and middleware patching for about 200 VMs across our plant sites was done by hand through tickets. It usually took two weekends per cycle, and missed steps caused recurring incidents.

Task: It was not formally my responsibility, but I was regularly pulled into the resulting incidents and could see the process was the root cause.

Action: I started small: a Python and Bash script for one site that ran pre-checks for disk, services and backups, applied patches, and ran post-checks with an automatic rollback if a health check failed. I ran it myself on non-critical VMs first. Once it was stable, I added staged rollout by site, a dry-run mode, and a simple report the ops team could review. Then I documented it and trained the ops engineers so they owned it.

Result: A patch cycle went from two weekends to a single evening, operational ticket volume dropped about 30%, and recurring infrastructure incidents went down about 25%. The automation was adopted across all sites.

Likely follow-ups:

- How did you get ops to trust the automation?
- What would you have done if a patch broke production?

Q5. Give an example of when you shipped a quick, imperfect solution and then iterated toward a better one.

Situation: When I joined Sprouts AI, on-call engineers were getting around 40 pages a week. Many were noise, so real problems were getting lost and MTTR was high.

Task: I wanted to cut page volume and MTTR quickly, without waiting for a full observability redesign.

Action: Week one, I did the grungy fix: I pulled 90 days of alert history and deleted or downgraded every alert that had never led to action. That alone cut pages noticeably. Next I defined SLOs for our critical paths and replaced threshold alerts with multi-window error-budget burn-rate alerts. Then I linked a runbook to every page and added dashboards that started at the symptom and drilled down to the component. Each step was reviewed against page data before the next.

Result: Paged incidents dropped 38% and MTTR dropped 45%, measured through our postmortems. On-call also became something engineers stopped dreading, which helped when I later brought newer engineers into the rotation.

Likely follow-ups:

- How did you decide which alerts were safe to delete?

- How do burn-rate alerts work?

Q6. Tell me about a time you came with a solution rather than just raising a problem.

Situation: During a production incident at Sprouts AI, we could see that overall latency was elevated but could not tell which tenants were affected. Customer support had to wait for us to grep logs.

Task: After the incident, the action item was simply 'improve tenant visibility', which had no owner and no plan. I decided to own it.

Action: The next day I built a rough per-tenant dashboard from metrics we already emitted, by adding a tenant label to the existing latency and error histograms and capping cardinality to the top tenants. I shared it with support and on-call that same week. Then I iterated: added per-tenant SLOs for our largest customers, alerts on per-tenant burn rate, and a status view support could read without engineering help.

Result: In the next incident we identified the affected tenants in minutes, and support proactively contacted them before they opened tickets. Per-tenant SLOs became part of our standard onboarding for new large tenants.

Likely follow-ups:

- How did you handle metrics cardinality?
- Who maintains it now?

3. Nail the Basics

What it means: Fundamentals over flash. Recognize when the basics are missing and fill the gaps. The path to advanced solutions runs through the basics, and progress is made even with incomplete information.

Interviewers look for: profiling before redesigning, getting correctness right (idempotency, timeouts, indexes), and resisting shiny solutions.

Q7. Tell me about a time when solving a problem required going back to fundamentals instead of a big new solution.

Situation: At Resilience Inc, a critical Java service had p99 latency that kept creeping up as traffic grew. The team's proposal was either to rewrite it in Go or to put a new caching tier in front of it, both multi-month efforts.

Task: I was asked to lead the performance work, and I wanted to understand where the time was actually going before we committed to a rewrite.

Action: I set up a repeatable load test that mirrored production traffic, then profiled. Three basic problems showed up: long GC pauses from humongous allocations on a large G1 heap, thread-pool starvation because blocking network I/O shared a pool with CPU-bound work, and a database connection pool that was too small, so requests queued. I tuned the GC settings and reduced large allocations, separated I/O and compute thread pools, right-sized connection pooling, and fixed network timeouts. I validated each change in isolation under load so we knew exactly what helped.

Result: p99 latency dropped 55% and throughput doubled, measured with APM telemetry, with no rewrite. We saved months of work and the load test became part of our release process.

Likely follow-ups:

- How did you identify humongous allocations?
- What would have made a rewrite the right call?

Q8. Describe a time you found that the basics were missing in a system and filled the gap.

Situation: At Resilience Inc, our Kafka pipeline persisting events to Cassandra and PostgreSQL was occasionally double-counting events after consumer rebalances and retries. Downstream teams had started adding their own de-duplication hacks.

Task: I owned the pipeline, so I needed to fix the correctness problem at its source instead of letting workarounds spread.

Action: I went back to delivery semantics. Kafka gives at-least-once delivery, so our writes had to be idempotent. I enabled idempotent producers, made every event carry a deterministic event ID, changed Cassandra writes to use that ID as part of the primary key so replays overwrote rather than duplicated, and used upserts with unique constraints in PostgreSQL. I also moved offset commits to after successful persistence. Then I wrote replay tests that killed consumers mid-batch to prove no duplicates or losses.

Result: Duplicates went to zero, downstream teams removed their hacks, and the pipeline scaled to over 2 billion events a day with under 5 seconds of end-to-end lag.

Likely follow-ups:

- Why not exactly-once transactions in Kafka?
- How did you test for data loss?

Q9. Tell me about a time you made progress without complete information or a perfect solution.

Situation: At Tata Steel, users across more than 40 plant sites complained that the equipment maintenance application was slow, but we had no APM tooling and no budget approved for it yet.

Task: I needed to improve performance with the data we did have, rather than waiting months for tooling.

Action: I used what Oracle Database already gave us: AWR reports and execution plans for the slowest screens. They showed full table scans on work-order queries and missing composite indexes, plus repeated lookups of reference data that rarely changed. I added and tested the right indexes, rewrote the worst SQL, and introduced a replicated Redis cache for reference data with clear expiry rules. I measured before and after per screen so we had proof.

Result: Data retrieval improved about 30% and overall response time about 40%, and that data helped justify the APM budget the following year.

Likely follow-ups:

- How did you choose what to cache?
- How did you handle cache consistency?

4. Expect and Embrace Change

What it means: Align quickly with new priorities, keep situational awareness, let go of outdated plans and processes, and treat change as a growth opportunity.

Interviewers look for: fast re-prioritization with clear communication, adapting without resentment, and using the change to improve things.

Q10. What do you do when priorities change quickly? Give an example.

From the OCI prep pack

Situation: At Resilience Inc, I was mid-sprint on a feature for our multi-tenant services when a critical vulnerability was disclosed in a library we used, and at the same time an external security audit was scheduled for the following month.

Task: The priority instantly became closing critical vulnerabilities within SLA and being audit-ready, while keeping stakeholders of the feature informed.

Action: I paused the feature and told the product owner the same day, with a new estimate. I inventoried every service by exposure to rank where to patch first, rolled patches through our pipeline with canaries, and verified with scans. Since we were already in this mode, I used the momentum to close related gaps the audit would check: encryption in transit with mTLS between services, encryption at rest, and least-privilege IAM roles. I kept a single tracker visible to security, product and management.

Result: We closed 100% of critical vulnerabilities within SLA and passed the audit. The feature slipped by one sprint, which stakeholders accepted because they knew early and understood why.

Likely follow-ups:

- How do you decide what to drop?
- How did you communicate the slip?

Q11. Tell me about a time you had to adapt to a significant change in technology or process.

Situation: At Tata Steel, we released quarterly with manual deployments and a long manual test cycle. Leadership decided to move to CI/CD, and many engineers and ops staff were skeptical.

Task: I volunteered to lead the transition for my team's services, which meant learning new tooling and changing how my own team worked.

Action: I built the pipeline incrementally: automated builds and unit tests first, then system-level test plans, then automated deployments to test environments, and finally production with approvals. I introduced code reviews and secure coding checks, and I paired with the four junior engineers I mentored so they could own parts of the pipeline. When ops worried about losing control, I added approval gates and rollback steps they designed.

Result: Release cycle time dropped 50% and escaped defects dropped 30%. The skeptics became advocates, and other teams copied the setup.

Likely follow-ups:

- What resistance did you face?

- What would you change about that rollout?

Q12. Describe a time when a plan you were committed to was changed and you had to realign quickly.

Situation: At Sprouts AI, my quarter was planned around throughput improvements. Then a major customer signed on with a data-residency requirement, and leadership reprioritized to bring up a deployment in a new OCI region within six weeks.

Task: I had to drop my planned work and help get a full data-plane stack running in a new region quickly and safely.

Action: I saw that our Terraform had region-specific values hardcoded in many places, which would have made this slow and error-prone. I refactored it into region-parameterized modules with separate state per region, added automated validation and drift detection, and wrote a region bring-up checklist covering capacity, replication, SLO dashboards and alerting. I built the region in stages with smoke and load tests at each stage before routing customer traffic.

Result: The new region was live in five weeks with no launch issues. More importantly, later regions took days instead of weeks. My throughput work resumed the next quarter, and I fed the hot-path learnings from the region build into it.

Likely follow-ups:

- How did you test a brand-new region?
- What did you defer and how did you decide?

5. Innovate Together

What it means: Empathy and respect, inclusion of diverse perspectives, challenging one's own biases, and enabling everyone on the team to reach their potential.

Interviewers look for: using others' ideas, crediting them, mentoring, and working with people who think differently.

Q13. Tell me about a time when bringing in different perspectives led to a better solution.

Situation: At Sprouts AI, we needed a load-shedding strategy so traffic spikes would not cause cascading failures. SRE wanted simple global rate limits, product wanted premium tenants never rejected, and those positions were in conflict.

Task: I was leading the design, and I wanted a solution everyone could stand behind, not just my own preference.

Action: I ran a design review and specifically invited the quieter voices, including a junior engineer who had been on call during the last surge. He suggested shedding by request criticality rather than by tenant, so background and batch work would be dropped first and interactive requests last. I had been thinking only in per-tenant terms, so I openly said his approach was better, and we combined it with per-tenant quotas. I asked him to co-own the implementation and to present it at our engineering review.

Result: The design held up through 5x traffic surges and network partitions with zero cascading failures, verified by brownout tests. The engineer gained visibility, and the review format became how we ran later designs.

Likely follow-ups:

- How do you draw out quieter team members?
- How did you resolve the SRE and product conflict?

Q14. Tell me about how you have helped other engineers grow.

Situation: At Sprouts AI, only a few senior engineers could handle serious incidents, which created burnout and a single point of failure for the team.

Task: I took on mentoring six engineers in operational excellence so they could confidently join primary on-call.

Action: I created a structured program: shadow shifts, then reverse-shadow shifts where they led and I backed them up, plus monthly game days where I injected failures in staging. I taught them how to read our dashboards from symptom to cause, and I had each of them write and present at least one postmortem. I gave specific feedback after every shift instead of generic praise.

Result: All six joined the primary rotation within about three months, two later led sev-2 incidents on their own, and it contributed to the 45% MTTR reduction. At Tata Steel I did something similar for four junior engineers who all became independent owners of services.

Likely follow-ups:

- How do you tailor mentoring to different people?

- What do you do when someone is struggling?

Q15. Describe a time you worked with people who had a very different background or way of working from yours.

Situation: At Tata Steel, our maintenance platform was used by plant engineers and technicians across 40+ sites. They were not software people and had been frustrated by earlier tools that were designed without them.

Task: I was building backend services and APIs for a new release and wanted it to actually work for them on the plant floor.

Action: Instead of relying only on written requirements, I visited two plants and watched technicians use the tool during shifts. I learned they needed fast lookups of equipment history on spotty networks, not the rich reports we had planned. I adjusted the API design to return compact responses, added caching for equipment reference data, and set up a monthly call with plant representatives to review changes before release.

Result: Response times improved about 40%, adoption went up, and the platform reached 99.99% uptime. The plant teams started bringing improvement ideas to us directly, which made our roadmap much better.

Likely follow-ups:

- What surprised you on the site visits?
- How did you balance their requests with technical constraints?

6. Take Risks, Remain Calm

What it means: Take calculated, data-driven risks; stay calm in unexpected situations and execute mitigation plans; learn from failure.

Interviewers look for: managed blast radius, clear abort criteria, calm incident leadership, and an honest failure story with a lesson.

Q16. Tell me about a calculated risk you took. How did you assess it?

Situation: At Sprouts AI, our resilience was mostly theoretical. We had retries, timeouts and failover in the design, but we had never deliberately tested them under failure. Some leaders were nervous about injecting faults into anything close to production.

Task: I believed not testing was the bigger risk, and I needed to prove failure-handling worked without causing an outage.

Action: I proposed a phased fault-injection and brownout testing program. Phase one ran in staging with production-like load. Each experiment had a written hypothesis, a limited blast radius, and automatic abort criteria tied to SLO metrics. Only after three clean staging cycles did we run small, scheduled game days on a single production cell with on-call ready. I presented the data from each phase to leadership before expanding.

Result: We found three latent issues before customers did: a retry storm between two services, a dependency call with no timeout, and failover that did not trigger when a leader was half-alive. After fixes, we sustained SLOs through 5x surges and network partitions with zero cascading failures.

Likely follow-ups:

- What were your abort criteria?
- How did you get leadership approval?

Q17. Describe a high-pressure production incident. How did you stay calm and lead the response?

Also answers: 'Describe a big obstacle and how you overcame it' (prep pack)

Situation: One night at Sprouts AI, a network partition between availability domains caused replicas in one storage cell to lose quorum. Writes started failing for a subset of tenants, and alerts were firing everywhere.

Task: I was incident commander. I had to restore service quickly without making it worse, since a rushed manual leader election during a partition risked split-brain and data loss.

Action: I first set up roles: one person on communications, one on the network team bridge, and me on decisions. Someone suggested forcing a leader election on the minority side; I said no because it could produce two leaders. Instead we shed non-critical traffic to protect healthy replicas, redirected reads to healthy nodes, and posted customer updates every 15 minutes. Once the network team confirmed the partition source and fixed routing, quorum came back on its own and we verified consistency with checksums before restoring full traffic.

Result: Service was fully restored in 38 minutes with no data loss. In the blameless postmortem we added partition-aware health checks and a runbook for quorum loss, and the same event later recovered automatically in minutes.

Likely follow-ups:

- What signals told you it was a partition?
- When would you have changed course?

Q18. Tell me about a risk you took that did not work out. What did you learn?

Situation: Early at Sprouts AI, I pushed to make automatic failover much more aggressive to cut our recovery time. I lowered the leader heartbeat timeout to two seconds.

Task: My goal was faster recovery, but I owned the consequences of the change.

Action: A week later, a combination of GC pauses and network jitter in one cell caused repeated false failovers, leadership flapping for about 12 minutes and elevated errors. I stayed calm, disabled automatic failover for that cell to stop the flapping, and confirmed it stabilized. Then I analyzed the data: the timeout was tighter than our real p99.9 pause time. I fixed it properly with a more adaptive failure detector, hysteresis so a node must fail several checks in a row, and a rate limit on failovers per cell. I wrote the postmortem myself and owned that the original change lacked a staged rollout.

Result: We still got recovery under 30 seconds, and we had zero flapping since. My lesson: a risky change needs guardrails and a staged rollout just like code, and the risk assessment has to use real tail data, not averages.

Likely follow-ups:

- How would you roll out a change like that today?
- How did your team react?

7. Own Without Ego

What it means: Take responsibility for your team, product and yourself. Admit mistakes first, welcome feedback, adopt the best idea regardless of source, and never say 'that's not my job'.

Interviewers look for: a real mistake you caused, taking orphaned problems, and changing your mind based on data.

Q19. Tell me about a mistake you made. How did you handle it?

Situation: At Tata Steel, I shipped a change that added Redis caching for work-order status to speed up the maintenance platform.

Task: I owned the change, the review and the release.

Action: My invalidation logic missed one update path, so for about three hours at two plants, closed work orders still showed as open, and technicians were dispatched to jobs that were already done. When the plant reported it, I immediately said it was likely my change, rolled back the caching, and confirmed data was correct from the database. I personally called the plant leads to explain and apologize. Then I fixed the root cause with write-through invalidation, a short TTL as a safety net, and tests for every update path, and I put the feature behind a flag for a staged re-release.

Result: The re-release had no issues and kept the performance gains. I also added a checklist item to our code reviews for cache invalidation paths. My lesson was to always ask 'what makes this data stale?' before adding a cache, and to ship caching changes behind a flag.

Likely follow-ups:

- How did you detect it?
- What would you do the same way again?

Q20. Tell me about a time you took ownership of something that was not your responsibility.

Situation: At Sprouts AI, an internal TLS certificate came within hours of expiring on a service that many data-plane components depended on. Nobody noticed until a warning showed up in logs, and no team clearly owned certificate rotation.

Task: It was not my area, but an expiry would have broken service-to-service traffic, so I decided to handle it rather than wait for an owner.

Action: I rotated the certificate immediately with another engineer reviewing. Then I inventoried all certificates across our services, found several more expiring within 60 days, and built automated rotation in our IaC pipeline with alerts at 30 and 7 days before expiry. I wrote a runbook and proposed that platform security formally own it, then handed it over with documentation.

Result: We had zero certificate-related incidents after that, and the inventory found two other near-misses. It reinforced for me that 'no owner' is an outage waiting to happen.

Likely follow-ups:

- How did you get the security team to own it?
- How do you balance this with your own work?

Q21. Tell me about a time someone else's idea was better than yours, or when you received difficult feedback.

Situation: At Sprouts AI, we needed a store for control-plane metadata. I proposed building a Raft-based replicated store, since I had built one in my Keystone project and knew the design well.

Task: I was leading the design review and wanted the right answer for the team, not my preferred one.

Action: A peer argued that our metadata write rate was low and a managed PostgreSQL setup with synchronous replicas would meet our durability and availability needs with far less operational risk. I asked for numbers, and the data supported him: our peak writes were orders of magnitude below what PostgreSQL could handle. I said in the review that his option was better and supported it. Separately, my manager told me my design docs were too long and decisions got lost, so I switched to one-page decision docs with options and a clear recommendation up front.

Result: We shipped about six weeks sooner and had one less complex system to operate. The shorter decision docs made reviews faster and are now the team template.

Likely follow-ups:

- When would building your own have been right?
- How do you respond when you disagree with feedback?

8. Earn Trust, Give Trust

What it means: Communicate openly and transparently, trust others to be responsible, learn from failures without blaming, and never use rank to win arguments.

Interviewers look for: delivering bad news early, delegating real ownership, and blameless postmortems.

Q22. Tell me about a time you had to deliver bad news to stakeholders.

Situation: At Sprouts AI, I was leading the replication redesign to improve throughput. Halfway through, load testing revealed that our quorum write path needed a deeper change than planned.

Task: We were going to miss the committed date by about three weeks, and product had made promises to customers based on it.

Action: I told my manager and the product lead the same week I knew, rather than hoping to catch up. I brought the load-test data showing why, and three options: slip the whole thing three weeks, ship the read-path improvements on time and the write path later, or cut scope. I recommended the phased option and was clear about the risk in each.

Result: They chose the phased plan. Customers got the read improvements on schedule, and the full redesign delivered three times the throughput with p99 under 40 ms. The product lead later told me he trusted my estimates more because I flagged problems early.

Likely follow-ups:

- Why not work extra hours to make the date?
- How did product react initially?

Q23. Tell me about a time you trusted someone with something important.

Situation: At Sprouts AI, we needed a canary rollout and automatic rollback system for our in-service updates. I could have built it myself quickly.

Task: I wanted to grow a mid-level engineer on my team and spread knowledge so I would not be a bottleneck.

Action: I gave her full ownership, including the design. I agreed on a couple of checkpoints, a design review and a review before first production use, and otherwise stayed out of the way. When she chose a different metric-comparison approach from what I would have used, I asked questions instead of overriding her, and her approach was simpler.

Result: She delivered it on time, and it became the system behind more than 300 in-service updates with zero customer-facing downtime. She is now the go-to owner for deployment safety, and I was freed to focus on the replication work.

Likely follow-ups:

- What would you have done if she was going off track?
- How do you decide what to delegate?

Q24. Tell me about a time something went wrong because of someone else's mistake. How did you handle it?

Situation: At Sprouts AI, a junior engineer pushed a configuration change that set a wrong timeout value, causing elevated errors in one region for about 20 minutes.

Task: I ran the postmortem. Some people wanted to know who made the mistake, and the engineer was clearly shaken.

Action: I ran it as a blameless postmortem and framed the question as 'why was it possible for one config value to cause this?' The answers were the real root causes: config changes had no validation and no staged rollout, unlike code. I talked privately with the engineer to make sure he knew he was not being blamed, and I asked him to help implement the fix. We added schema validation and safe ranges for configs and routed config changes through the same canary process as code.

Result: We had no further config-caused severe incidents, and the engineer became one of the strongest contributors to our change-safety work. Others also became more willing to report near-misses.

Likely follow-ups:

- How do you keep accountability in a blameless culture?
- What did leadership think?

9. Take Pride in Your Work

What it means: Strive for excellence, communicate goals, change the work or yourself when you are not proud of it, and invest in things that help you excel and delight customers.

Interviewers look for: high quality bar, measurable outcomes, and going beyond the minimum.

Q25. What work are you most proud of and why?

Situation: At Sprouts AI, I led the design of the Java and Go data-plane services for a multi-tenant platform spanning more than 12,000 nodes on OCI.

Task: The bar was 99.99% availability while still shipping changes frequently, which are usually in tension.

Action: I defined the scalability, elasticity and durability requirements up front, and designed for in-service updates: redundancy across fault domains, automatic failover, and recovery-oriented design. I built Terraform-based IaC with canary rollouts and automatic rollback, and tied every rollout step to SLO checks. I also validated the design with load, fault-injection and brownout tests instead of trusting it on paper.

Result: We have sustained 99.99% availability on our SLO dashboards while shipping more than 300 in-service updates with zero customer-facing downtime. I am proud of it because reliability and speed reinforced each other instead of trading off.

Likely follow-ups:

- What part of the design would you change?
- How do you measure availability?

Q26. Tell me about a time you raised the quality bar beyond what was required.

Situation: At Resilience Inc, test coverage on our services was low, integration tests were flaky, and deployments took nearly an hour, so engineers avoided deploying often.

Task: Nothing formally required a fix, but I was not comfortable shipping critical services with that safety net.

Action: I built system-level test automation and fault-injection suites that ran against real dependencies in containers. I quarantined flaky tests and fixed their root causes one by one instead of adding retries. I parallelized the pipeline, moved it to IaC, and added automated rollbacks so a bad deploy undid itself.

Result: Automated coverage rose to 85% and deployment time fell 60%. Engineers deployed smaller changes more often, which lowered risk overall.

Likely follow-ups:

- How did you get time allocated for this?
- What was the hardest flaky test to fix?

Q27. Tell me about a time you were not satisfied with your work and changed it.

Situation: In my own time I built Keystone, a replicated metadata store in Go using Raft, to deepen my distributed-systems skills.

Task: The first version passed all its unit tests, but I did not trust that it was actually correct under real failures.

Action: I built a fault-injection harness that randomly partitioned nodes, crashed them and delayed messages, while a checker verified the history was linearizable. It quickly found a real bug: a deposed leader could serve stale reads during a partition. I fixed it with a read-index approach so reads confirm leadership with a quorum, and added snapshot recovery tests.

Result: Keystone is now correct across 41 injected partition and crash scenarios. Those testing habits carried directly into how I designed fault-injection testing at Sprouts AI.

Likely follow-ups:

- Explain the stale-read bug in detail.
- How does a read index work?

10. Challenge Ideas, Champion Execution

What it means: Ask why, test ideas with rigor and data, create clarity, own decisions, and then drive effective implementation.

Interviewers look for: data-backed challenges, comparing alternatives, and following through to delivery after the debate.

Q28. There is more than one way to solve a problem. Give an example from your recent work.

From the OCI prep pack

Situation: At Sprouts AI, our high-throughput storage path needed to handle about three times its traffic within two quarters while keeping p99 retrieval latency under 40 ms.

Task: I owned the design and needed to choose an approach.

Action: I laid out three options. One: scale out horizontally, which was simple but would roughly triple infrastructure cost. Two: relax to eventual consistency, which would be fast but break guarantees some tenants relied on. Three: optimize the data path itself. I built a load-test harness and prototyped option three: batching and pipelining quorum writes, serving reads from a lease-holding leader to avoid extra round trips, and reducing synchronization contention in our distributed state. I presented the data for all three options.

Result: We went with option three, tripled throughput with p99 under 40 ms, kept strong consistency, and kept infrastructure cost roughly flat. The comparison doc also became a reference for later capacity planning.

Likely follow-ups:

- What are the risks of leader leases?
- How did you make the load test realistic?

Q29. Tell me about a time you challenged a widely accepted idea or decision.

Situation: At Sprouts AI, after several incidents involving flaky dependencies, the team agreed to add retries at every layer of the call chain.

Task: I believed this would make things worse under load and wanted to show it with data rather than opinion.

Action: I did the math: with three layers each retrying three times, a single failing request could become 27 calls at the bottom, so a small outage could turn into a retry storm. I reproduced this in a brownout test and showed the graphs. Then I proposed an alternative: retry at only one layer, use exponential backoff with jitter, cap retries with a retry budget per client, set explicit timeouts at every hop, and add circuit breakers. After the team agreed, I wrote the shared client library so everyone used it consistently.

Result: During later 5x traffic surges and partial outages we saw no retry storms and zero cascading failures.

Likely follow-ups:

- How do retry budgets work?
- How did people react to being challenged?

Q30. Tell me about a time you drove a decision to completion after a long debate.

Situation: At Sprouts AI, the team had debated for weeks whether to standardize on Terraform, another IaC tool, or our in-house scripts. Meanwhile, infrastructure changes were still manual and risky.

Task: I wanted to end the debate with a clear decision and then actually get it done.

Action: I proposed a one-week, time-boxed spike with agreed criteria: multi-region support, state management, drift detection, team skills and OCI provider maturity. We built the same small stack in each option. I wrote a one-page decision doc recommending Terraform, and once we agreed I created the migration plan: shared modules, separate state per region, CI plan and apply with reviews, and drift detection. I tracked progress weekly and migrated the riskiest components myself.

Result: All our infrastructure was in code within one quarter. That foundation enabled more than 300 safe in-service updates and later let us stand up a new region in five weeks.

Likely follow-ups:

- What did people who disagreed do afterwards?
- How did you handle existing manual resources?

Prep-pack sample questions: quick map

Question in the OCI prep pack	Use this answer
Tell me about a time you had to prioritize your customers' needs…	Q1: hot-shard fix for enterprise go-live
Describe a work situation where you faced a big obstacle.	Q17: network partition incident (backup: Q7, GC tuning instead of a rewrite)
There is more than one way to solve a problem…	Q28: three options for 3x throughput
What do you do when priorities change quickly?	Q10: critical CVE and audit (backup: Q12, new OCI region)

Questions to ask your interviewers

- What are the biggest reliability or scale challenges for this team over the next 12 months?
- How does the team define and review SLOs, and who owns the error budget decisions?
- How are in-service updates and region build-outs handled today, and where is automation still missing?
- What does success look like for a Principal engineer here after six months?
- How do Principal engineers influence architecture across teams in OCI?

Final reminders

- Do not share anything confidential from current or past employers; OCI's pack calls this out explicitly. Use generic names (for example, "a large enterprise tenant").
- Each answer needs a clear beginning, middle and end. If you run long, cut Situation detail, never the Result.
- If you do not have a perfect example, say so and give the closest real one. A true story with honest gaps beats a polished one that falls apart under follow-up.
- Have one failure story ready (Q18 or Q19). OCI explicitly values learning from failure.